

## How to create a project in STM32CubeIDE :

### Open STM32CubeIDE.

Confirm your default workspace when prompted (if auto-selected, wait for the dashboard to load).

Initiate the Project Wizard Click File from the top menu bar.

Navigate to **New > STM32 Project**. (Wait a few seconds for the target selector network database to launch.)

### Create STM32 empty project

Target MCU Selection Select the MCU/MPU Selector tab (default tab).

Type **STM32F446RETx** into the **Commercial Part Number** search box.

Click on the exact entry **STM32F446RETx** in the lower results table.

Click the **Next button**.

Project Naming & Structure Setup

Input your preferred name in the **Project Name** field.

Click **Finish**.

see folder :

**src/main.c**

this main.c only modify for Lab experiment , others all remain same .

**For I2C Master-Slave data communication experiment , follow this sequence carefully:**

**We need :**

- 1) Two STM32F446RE Nucleo Board**
- 2) Two Wire required**

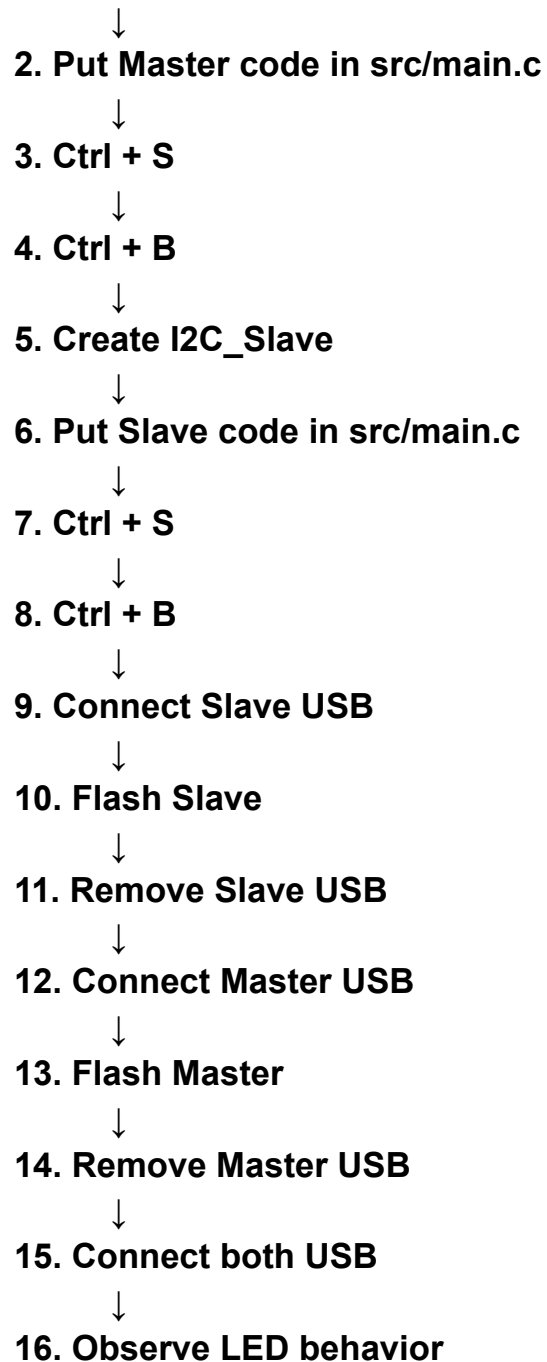
**First check the wiring:**

**Master D15 / (SCL) (CN9 channel) ————— Slave D15 / (SCL) ----(CN9 channel)**

**Master D14 / SDA (CN9 channel) ————— Slave D14 / SDA ----(CN9 channel)**

Create Two project :

**1. Create I2C\_Master**

- 
- ```
graph TD; A[ ] --> B[2. Put Master code in src/main.c]; B --> C[3. Ctrl + S]; C --> D[4. Ctrl + B]; D --> E[5. Create I2C_Slave]; E --> F[6. Put Slave code in src/main.c]; F --> G[7. Ctrl + S]; G --> H[8. Ctrl + B]; H --> I[9. Connect Slave USB]; I --> J[10. Flash Slave]; J --> K[11. Remove Slave USB]; K --> L[12. Connect Master USB]; L --> M[13. Flash Master]; M --> N[14. Remove Master USB]; N --> O[15. Connect both USB]; O --> P[16. Observe LED behavior];
```
- ↓
  2. Put Master code in src/main.c
  - ↓
  3. Ctrl + S
  - ↓
  4. Ctrl + B
  - ↓
  5. Create I2C\_Slave
  - ↓
  6. Put Slave code in src/main.c
  - ↓
  7. Ctrl + S
  - ↓
  8. Ctrl + B
  - ↓
  9. Connect Slave USB
  - ↓
  10. Flash Slave
  - ↓
  11. Remove Slave USB
  - ↓
  12. Connect Master USB
  - ↓
  13. Flash Master
  - ↓
  14. Remove Master USB
  - ↓
  15. Connect both USB
  - ↓
  16. Observe LED behavior

### **The observed behavior is:**

#### **Initial:**

Master LED = ON

Slave LED = ON

#### **Master button pressed and held:**

Master LED = OFF  
Slave LED = OFF

### **Master button released:**

Master LED = ON  
Slave LED = OFF

### **I2C\_Master Code :**

```
typedef struct {  
    volatile unsigned int MODER;  
    volatile unsigned int OTYPER;  
    volatile unsigned int OSPEEDR;  
    volatile unsigned int PUPDR;  
    volatile unsigned int IDR;  
    volatile unsigned int ODR;  
    volatile unsigned int BSRR;  
    volatile unsigned int LCKR;  
    volatile unsigned int AFR[2];  
} GPIO_TypeDef;
```

```
typedef struct {  
    volatile unsigned int CR;  
    volatile unsigned int PLLCFGR;  
    volatile unsigned int CFGR;  
    volatile unsigned int CIR;  
    volatile unsigned int AHB1RSTR;  
    volatile unsigned int AHB2RSTR;  
    volatile unsigned int AHB3RSTR;  
    unsigned int RESERVED0;  
    volatile unsigned int APB1RSTR;  
    volatile unsigned int APB2RSTR;  
    unsigned int RESERVED1[2];  
    volatile unsigned int AHB1ENR;  
    volatile unsigned int AHB2ENR;  
    volatile unsigned int AHB3ENR;  
    unsigned int RESERVED2;  
    volatile unsigned int APB1ENR;  
    volatile unsigned int APB2ENR;  
} RCC_TypeDef;
```

```
typedef struct {  
    volatile unsigned int CR1;
```

```
volatile unsigned int CR2;  
volatile unsigned int OAR1;  
volatile unsigned int OAR2;  
volatile unsigned int DR;  
volatile unsigned int SR1;  
volatile unsigned int SR2;  
volatile unsigned int CCR;  
volatile unsigned int TRISE;  
volatile unsigned int FLTR;  
} I2C_TypeDef;
```

```
#define RCC_BASE    0x40023800U  
#define GPIOA_BASE  0x40020000U  
#define GPIOB_BASE  0x40020400U  
#define GPIOC_BASE  0x40020800U  
#define I2C1_BASE   0x40005400U
```

```
#define RCC    ((RCC_TypeDef *)RCC_BASE)  
#define GPIOA  ((GPIO_TypeDef *)GPIOA_BASE)  
#define GPIOB  ((GPIO_TypeDef *)GPIOB_BASE)  
#define GPIOC  ((GPIO_TypeDef *)GPIOC_BASE)  
#define I2C1   ((I2C_TypeDef *)I2C1_BASE)
```

```
#define RCC_AHB1ENR_GPIOAEN (1U << 0)  
#define RCC_AHB1ENR_GPIOBEN (1U << 1)  
#define RCC_AHB1ENR_GPIOCEN (1U << 2)  
#define RCC_APB1ENR_I2C1EN  (1U << 21)
```

```
#define I2C_CR1_PE    (1U << 0)  
#define I2C_CR1_START (1U << 8)  
#define I2C_CR1_STOP  (1U << 9)  
#define I2C_CR1_ACK   (1U << 10)  
#define I2C_CR1_SWRST (1U << 15)
```

```
#define I2C_SR1_SB    (1U << 0)  
#define I2C_SR1_ADDR  (1U << 1)  
#define I2C_SR1_BTF   (1U << 2)  
#define I2C_SR1_STOPF (1U << 4)  
#define I2C_SR1_RXNE  (1U << 6)  
#define I2C_SR1_TXE   (1U << 7)  
#define I2C_SR1_AF    (1U << 10)
```

```
#define SLAVE_ADDR  0x30U  
#define I2C_TIMEOUT 100000U
```

```
volatile unsigned char i2c_last_error = 0; /* inspect this with the debugger */
```

```
static void GPIO_Init(void);  
static void I2C1_Init(void);  
static unsigned char I2C1_Start(void);  
static unsigned char I2C1_SendAddress(unsigned char addr);  
static unsigned char I2C1_WriteByte(unsigned char data);  
static void I2C1_Stop(void);  
static void I2C1_Recover(void);  
static unsigned char Button_Pressed(void);  
static void delay(volatile unsigned int t);  
static void LED_On(void);  
static void LED_Off(void);
```

```
int main(void)  
{  
    GPIO_Init();  
    I2C1_Init();  
  
    LED_On();  
  
    unsigned char last_button = 0;  
  
    while (1)  
    {  
        unsigned char button_now = Button_Pressed();  
  
        if (button_now && !last_button)  
        {  
            /* --- Press edge --- */  
            LED_Off();  
  
            i2c_last_error = 0;  
  
            if (!I2C1_Start())  
            {  
                i2c_last_error = 1;  
                I2C1_Recover();  
            }  
            else if (!I2C1_SendAddress(SLAVE_ADDR))  
            {  
                /* i2c_last_error already set inside I2C1_SendAddress */  
                I2C1_Recover();  
            }  
        }  
    }  
}
```

```

    }
    else if (!I2C1_WriteByte(0x00))
    {
        I2C1_Recover();
    }
    else
    {
        I2C1_Stop();
    }
}
else if (!button_now && last_button)
{
    /* --- Release edge --- */
    LED_On();
}

last_button = button_now;
delay(30000);
}
}

```

```

static void GPIO_Init(void)
{
    RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN | RCC_AHB1ENR_GPIOBEN |
    RCC_AHB1ENR_GPIOCEN;

    GPIOA->MODER  &= ~(3U << (5 * 2));
    GPIOA->MODER  |= (1U << (5 * 2));
    GPIOA->OTYPER &= ~(1U << 5);
    GPIOA->OSPEEDR |= (3U << (5 * 2));

    GPIOC->MODER &= ~(3U << (13 * 2));
    GPIOC->PUPDR &= ~(3U << (13 * 2));

    GPIOB->MODER &= ~((3U << (8 * 2)) | (3U << (9 * 2)));
    GPIOB->MODER |= ((2U << (8 * 2)) | (2U << (9 * 2)));

    GPIOB->OTYPER |= (1U << 8) | (1U << 9);

    GPIOB->OSPEEDR |= (3U << (8 * 2)) | (3U << (9 * 2));

    GPIOB->PUPDR &= ~((3U << (8 * 2)) | (3U << (9 * 2)));
    GPIOB->PUPDR |= ((1U << (8 * 2)) | (1U << (9 * 2)));
}

```

```

    GPIOB->AFR[1] &= ~((0xFU << ((8 - 8) * 4)) | (0xFU << ((9 - 8) * 4)));
    GPIOB->AFR[1] |= ((4U << ((8 - 8) * 4)) | (4U << ((9 - 8) * 4)));
}

```

```

static void I2C1_Init(void)
{
    RCC->APB1ENR |= RCC_APB1ENR_I2C1EN;

    I2C1->CR1 |= I2C_CR1_SWRST;
    I2C1->CR1 &= ~I2C_CR1_SWRST;

    I2C1->CR2 = 16;

    I2C1->CCR = 80;
    I2C1->TRISE = 17;

    I2C1->CR1 |= I2C_CR1_PE;
    I2C1->CR1 |= I2C_CR1_ACK;
}

```

```

static unsigned char I2C1_Start(void)
{
    unsigned int timeout = I2C_TIMEOUT;
    I2C1->CR1 |= I2C_CR1_START;
    while (!(I2C1->SR1 & I2C_SR1_SB))
    {
        if (--timeout == 0) return 0;
    }
    return 1;
}

```

```

static unsigned char I2C1_SendAddress(unsigned char addr)
{
    unsigned int timeout = I2C_TIMEOUT;
    I2C1->DR = (unsigned char)((addr << 1) | 0U);

    while (!(I2C1->SR1 & (I2C_SR1_ADDR | I2C_SR1_AF)))
    {
        if (--timeout == 0) { i2c_last_error = 3; return 0; }
    }

    if (I2C1->SR1 & I2C_SR1_AF)
    {
        I2C1->SR1 &= ~I2C_SR1_AF; /* clear AF flag */
    }
}

```

```

        i2c_last_error = 2;    /* address was NACKed */
        return 0;
    }

    (void)I2C1->SR1;
    (void)I2C1->SR2; /* clears ADDR */
    return 1;
}

static unsigned char I2C1_WriteByte(unsigned char data)
{
    unsigned int timeout = I2C_TIMEOUT;
    while (!(I2C1->SR1 & I2C_SR1_TXE))
    {
        if (--timeout == 0) { i2c_last_error = 4; return 0; }
    }

    I2C1->DR = data;

    timeout = I2C_TIMEOUT;
    while (!(I2C1->SR1 & I2C_SR1_BTF))
    {
        if (--timeout == 0) { i2c_last_error = 5; return 0; }
    }
    return 1;
}

static void I2C1_Stop(void)
{
    I2C1->CR1 |= I2C_CR1_STOP;
}

static void I2C1_Recover(void)
{
    I2C1->CR1 |= I2C_CR1_STOP;
    delay(2000);
    I2C1_Init(); /* SWRST toggle + full re-init */
}

static unsigned char Button_Pressed(void)
{
    return ((GPIOC->IDR & (1U << 13)) == 0U);
}

```



```

static void LED_On(void)
{
    GPIOA->BSRR = (1U << 5);
}

static void LED_Off(void)
{
    GPIOA->BSRR = (1U << (5 + 16));
}

static void delay(volatile unsigned int t)
{
    while (t--) { }
}

```

## **I2C\_Slave Code :**

```

typedef struct {
    volatile unsigned int MODER;
    volatile unsigned int OTYPER;
    volatile unsigned int OSPEEDR;
    volatile unsigned int PUPDR;
    volatile unsigned int IDR;
    volatile unsigned int ODR;
    volatile unsigned int BSRR;
    volatile unsigned int LCKR;
    volatile unsigned int AFR[2];
} GPIO_TypeDef;

```

```

typedef struct {
    volatile unsigned int CR;
    volatile unsigned int PLLCFGR;
    volatile unsigned int CFGR;
    volatile unsigned int CIR;
    volatile unsigned int AHB1RSTR;
    volatile unsigned int AHB2RSTR;
    volatile unsigned int AHB3RSTR;
    unsigned int RESERVED0;
    volatile unsigned int APB1RSTR;
    volatile unsigned int APB2RSTR;
    unsigned int RESERVED1[2];
}

```

```
volatile unsigned int AHB1ENR;
volatile unsigned int AHB2ENR;
volatile unsigned int AHB3ENR;
unsigned int RESERVED2;
volatile unsigned int APB1ENR;
volatile unsigned int APB2ENR;
} RCC_TypeDef;
```

```
typedef struct {
    volatile unsigned int CR1;
    volatile unsigned int CR2;
    volatile unsigned int OAR1;
    volatile unsigned int OAR2;
    volatile unsigned int DR;
    volatile unsigned int SR1;
    volatile unsigned int SR2;
    volatile unsigned int CCR;
    volatile unsigned int TRISE;
    volatile unsigned int FLTR;
} I2C_TypeDef;
```

```
#define RCC_BASE    0x40023800U
#define GPIOA_BASE  0x40020000U
#define GPIOB_BASE  0x40020400U
#define I2C1_BASE   0x40005400U
```

```
#define RCC    ((RCC_TypeDef *)RCC_BASE)
#define GPIOA  ((GPIO_TypeDef *)GPIOA_BASE)
#define GPIOB  ((GPIO_TypeDef *)GPIOB_BASE)
#define I2C1   ((I2C_TypeDef *)I2C1_BASE)
```

```
#define RCC_AHB1ENR_GPIOAEN (1U << 0)
#define RCC_AHB1ENR_GPIOBEN (1U << 1)
#define RCC_APB1ENR_I2C1EN  (1U << 21)
```

```
#define I2C_CR1_PE    (1U << 0)
#define I2C_CR1_ACK   (1U << 10)
#define I2C_CR1_SWRST (1U << 15)
```

```
#define I2C_SR1_ADDR  (1U << 1)
#define I2C_SR1_STOPF (1U << 4)
#define I2C_SR1_RXNE  (1U << 6)
```

```
#define SLAVE_ADDR  0x30U
```

```

#define I2C_TIMEOUT 100000U

volatile unsigned char i2c_slave_last_error = 0; /* inspect with debugger */

static void GPIO_Init(void);
static void I2C1_Slave_Init(void);
static void LED_On(void);
static void LED_Off(void);

int main(void)
{
    GPIO_Init();
    I2C1_Slave_Init();

    LED_On();

    while (1)
    {
        /* Normal idle wait - blocks here until a master addresses us.
           This is expected behavior, not a bug (see header comment). */
        while (!(I2C1->SR1 & I2C_SR1_ADDR)) { }
        (void)I2C1->SR1;
        (void)I2C1->SR2; /* clears ADDR */

        /* From here on, the address WAS matched, so a stall would be
           a genuine fault - these two stages have timeouts. */
        unsigned int timeout = I2C_TIMEOUT;
        while (!(I2C1->SR1 & I2C_SR1_RXNE))
        {
            if (--timeout == 0)
            {
                i2c_slave_last_error = 1;
                goto recover;
            }
        }
        (void)I2C1->DR; /* byte received -> turn LED off */
        LED_Off();
        i2c_slave_last_error = 0;

        timeout = I2C_TIMEOUT;
        while (!(I2C1->SR1 & I2C_SR1_STOPF))
        {
            if (--timeout == 0)
            {

```

```

        i2c_slave_last_error = 2;
        goto recover;
    }
}
(void)I2C1->SR1;
I2C1->CR1 |= I2C_CR1_PE; /* clears STOPF */

continue;

recover:
    I2C1_Slave_Init(); /* SWRST toggle + full re-init, go back to waiting */
}
}

static void GPIO_Init(void)
{
    RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN | RCC_AHB1ENR_GPIOBEN;

    GPIOA->MODER  &= ~(3U << (5 * 2));
    GPIOA->MODER  |= (1U << (5 * 2));
    GPIOA->OTYPER &= ~(1U << 5);
    GPIOA->OSPEEDR |= (3U << (5 * 2));

    GPIOB->MODER &= ~((3U << (8 * 2)) | (3U << (9 * 2)));
    GPIOB->MODER |= ((2U << (8 * 2)) | (2U << (9 * 2)));

    GPIOB->OTYPER |= (1U << 8) | (1U << 9);

    GPIOB->OSPEEDR |= (3U << (8 * 2)) | (3U << (9 * 2));

    GPIOB->PUPDR &= ~((3U << (8 * 2)) | (3U << (9 * 2)));
    GPIOB->PUPDR |= ((1U << (8 * 2)) | (1U << (9 * 2)));

    GPIOB->AFR[1] &= ~((0xFU << ((8 - 8) * 4)) | (0xFU << ((9 - 8) * 4)));
    GPIOB->AFR[1] |= ((4U << ((8 - 8) * 4)) | (4U << ((9 - 8) * 4)));
}

static void I2C1_Slave_Init(void)
{
    RCC->APB1ENR |= RCC_APB1ENR_I2C1EN;

    I2C1->CR1 |= I2C_CR1_SWRST;
    I2C1->CR1 &= ~I2C_CR1_SWRST;

```

```
I2C1->CR2 = 16;
```

```
I2C1->OAR1 = 0;
```

```
I2C1->OAR1 |= (1U << 14);
```

```
I2C1->OAR1 |= (SLAVE_ADDR << 1);
```

```
I2C1->CCR = 80;
```

```
I2C1->TRISE = 17;
```

```
I2C1->CR1 |= I2C_CR1_PE;
```

```
I2C1->CR1 |= I2C_CR1_ACK;
```

```
}
```

```
static void LED_On(void)
```

```
{
```

```
    GPIOA->BSRR = (1U << 5);
```

```
}
```

```
static void LED_Off(void)
```

```
{
```

```
    GPIOA->BSRR = (1U << (5 + 16));
```

```
}
```